# ALU WITH TESTBENCH

Submitted by

**Mohammed Sinan**

# 1. Introduction

An Arithmetic Logic Unit (ALU) is a combinational digital circuit that performs arithmetic and logical operations on binary data. It is an essential part of a processor datapath. The ALU receives two operands and a control signal that selects the required operation.

This project describes a 4-bit ALU using Verilog Hardware Description Language (HDL). A separate testbench applies input combinations and checks the outputs through simulation. The design is intended as an educational example; the supplied project brief does not prescribe an operand width or operation encoding.

# 2. Objectives

• Understand the basic architecture and operation of an ALU.
• Design a 4-bit ALU using Verilog RTL.
• Develop a testbench to verify arithmetic and logical functions.
• Observe the output signals using a waveform viewer.
• Compare simulation results with expected values.

# 3. ALU Architecture and Operations

Inputs: A[3:0], B[3:0], and ALU_Sel[2:0]. Outputs: Result[3:0] and Cout. The ALU is combinational, so its outputs respond to changes in the operands or operation-select input.

| ALU_Sel | Operation | Description |
|---|---|---|
| 000 | ADD | Result = A + B; Cout is carry-out |
| 001 | SUB | Result = A – B; Cout = 1 means no borrow |
| 010 | AND | Bitwise A AND B |
| 011 | OR | Bitwise A OR B |
| 100 | XOR | Bitwise A exclusive-OR B |
| 101 | NOT A | Bitwise complement of A |
| 110 | PASS A | Result = A |
| 111 | PASS B | Result = B |

For subtraction, Cout is defined as 1 when A ≥ B (no borrow), and 0 when A < B (borrow).

# 4. Verilog RTL Design

### *File name: alu.v*

```
module alu (
    input  wire [3:0] A,
    input  wire [3:0] B,
    input  wire [2:0] ALU_Sel,
    output reg  [3:0] Result,
    output reg        Cout
);
    always @(*) begin
        Result = 4'b0000;
        Cout   = 1'b0;
        case (ALU_Sel)
            3'b000: {Cout, Result} = A + B;
            3'b001: begin
                Result = A - B;
                Cout = (A >= B); // 1 = no borrow
            end
            3'b010: Result = A & B;
            3'b011: Result = A | B;
            3'b100: Result = A ^ B;
            3'b101: Result = ~A;
            3'b110: Result = A;
            3'b111: Result = B;
            default: begin
```

```
                Result = 4'b0000;
                Cout = 1'b0;
            end
        endcase
    end
endmodule
```

# 5. Verilog Testbench

The testbench instantiates the ALU, applies test vectors, prints the outputs, and creates a VCD dump file for waveform inspection.

### *File name: alu_tb.v*

```
`timescale 1ns/1ps
module alu_tb;
    reg  [3:0] A, B;
    reg  [2:0] ALU_Sel;
    wire [3:0] Result;
    wire Cout;

    alu uut (
        .A(A), .B(B), .ALU_Sel(ALU_Sel),
        .Result(Result), .Cout(Cout)
    );

    initial begin
        $dumpfile("alu_wave.vcd");
        $dumpvars(0, alu_tb);

        A = 4'd9; B = 4'd6;

        ALU_Sel = 3'b000; #10;
        $display("ADD: A=%d B=%d Result=%d Cout=%b", A,B,Result,Cout);
        ALU_Sel = 3'b001; #10;
        $display("SUB: A=%d B=%d Result=%d Cout=%b", A,B,Result,Cout);
        ALU_Sel = 3'b010; #10;
        $display("AND: Result=%b", Result);
        ALU_Sel = 3'b011; #10;
        $display("OR : Result=%b", Result);
        ALU_Sel = 3'b100; #10;
        $display("XOR: Result=%b", Result);
        ALU_Sel = 3'b101; #10;
        $display("NOT: Result=%b", Result);
        ALU_Sel = 3'b110; #10;
        $display("PASS A: Result=%b", Result);
        ALU_Sel = 3'b111; #10;
        $display("PASS B: Result=%b", Result);

        A = 4'd3; B = 4'd5;
        ALU_Sel = 3'b001; #10;
        $display("SUB borrow: Result=%d Cout=%b", Result,Cout);
        $finish;
    end
endmodule
```

# 6. Simulation Results

The following expected values correspond to the testbench vectors. A = 9 and B = 6 are used for the first eight operations. The final vector uses A = 3 and B = 5 for subtraction.

| Select | Operation | Result (decimal) | Cout |
|---|---|---|---|
| 000 | ADD | 15 | 0 |
| 001 | SUB | 3 | 1 (no borrow) |
| 010 | AND | 0 | 0 |
| 011 | OR | 15 | 0 |
| 100 | XOR | 15 | 0 |
| 101 | NOT A | 6 | 0 |
| 110 | PASS A | 9 | 0 |
| 111 | PASS B | 6 | 0 |

| Select | Operation | Result (decimal) | Cout |
|---|---|---|---|
| 001 | SUB (3 – 5) | 14 | 0 (borrow) |

## Timing waveform

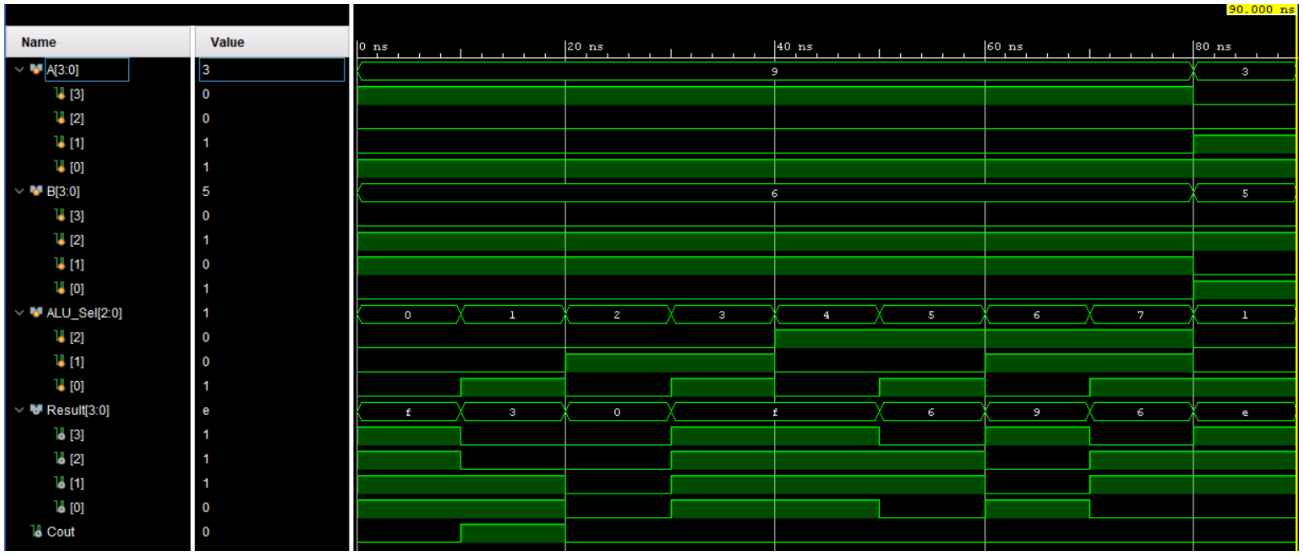



Figure: ALU testbench simulation waveform showing input signals, operation select, result, and carry-out.

# 7. Simulation Procedure

Example commands using Icarus Verilog:

```
iverilog -o alu_sim alu.v alu_tb.v
vvp alu_sim
gtkwave alu_wave.vcd
```

In the waveform viewer, inspect A[3:0], B[3:0], ALU_Sel[2:0], Result[3:0], and Cout. Confirm that the output corresponds to the operation selected at each time interval.

# 8. Applications

• Arithmetic and logical processing in processor datapaths.
• Data manipulation in microcontrollers and digital systems.
• Address and index calculations.
• Learning RTL design and functional verification.

# 9. Conclusion

A 4-bit ALU was designed using Verilog HDL and verified with a separate testbench. The ALU supports addition, subtraction, AND, OR, XOR, NOT, and pass-through operations. Representative input vectors were applied, and the expected outputs were tabulated. The VCD waveform process provides a way to inspect signal changes and verify the combinational behavior.

# References

1. Supplied minor project brief: “Minor Project Submission – ALU with Testbench.”
2. IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364.
3. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition.

**Submission reminder:** Submit alu.v, alu_tb.v, and the simulator-generated waveform/dump file. The timing diagram included here is illustrative; attach a real waveform screenshot from your simulator if your instructor requires one.